

Facilitator answer key — Holocron capstone worked solution.

Facilitator-only — lives in the facilitator folder of every repo; do not hand to participants. Part of the internally-consistent set in this folder (see README.md).

DP-light — Holocron release 1 (delivery tool: Azure DevOps)

Scope: the R1 slice (CS1–7, CS11–12, CS15). Out: aliases, advanced reviewer config, export/import, rollback.

1. Outcome map

Outcomes move the world; outputs (shipped stories) are how we move them. Each leading indicator is measurable within 7 days; each guardrail stops us gaming it.

#	Outcome	Leading indicator (≤7d)	Guardrail
O1 primary	Time to change a published UI string drops from days/weeks to minutes (no deploy)	p50 author→consumed lead time ≤ 15 min, daily	change must reach apps within propagation SLO (≤60s) — fast-to-publish-but-stale doesn't count
O2	Engineering hours on text-only changes fall toward zero	text-change code deploys/week → 0; # strings edited by non-engineers ↑	reject-on-publish + post-publish hot-fix rates stay flat
O3	Consuming apps adopt Holocron instead of hardcoding	# apps live on delivery API ↑; # namespaces served ↑	new apps still hardcode < X% — real integration, not a stub
O4 co- unter	We are NOT trading speed for governance	strings needing legal review that published <i>without</i> it = 0	(the guardrail for the whole plan)

2. Backlog skeleton (ADO)

One **Epic** → four **Features** → representative **User Stories**. Tag CSn traces to the PRD; R1 = release-1 slice. **Epic**: Holocron R1 — governed string lifecycle, no-deploy publishing · Holocron\R1

Feature	User Story	SP	Tags
F1 Authoring	US-101 Create/edit a source string in a product I own	5	CS3, CS4
	US-102 Publish a string; every version retained with author/timestamp	5	CS4, CS5, Audit
	US-103 Admin sets up a product + assigns Content Owners	3	CS1, CS2

Feature	User Story	SP	Tags
F2 Review	US-201 Request review so legal/compliance can approve before publish	3	CS11
	US-202 Reviewer completes a review (approve/reject + note), recorded in audit	3	CS12, Audit
F3 Delivery	US-301 Consuming app fetches strings by app+namespace over the delivery API	8	CS7
	US-302 App always gets an en-US fallback when a locale value is missing	3	CS7, Fallback
	US-303 Search strings across products	3	CS6
F4 Translation	US-401 Add/edit a locale variant of a published key	5	CS15

Immutable keys and the Redis cache are cross-cutting constraints (TCD/TMD-light), realized inside US-102/US-301, not separate stories.

3. Tracking plan (ADO)

Output	Outcome	Indicator	ADO instrument
US-102 publish + audit	O1	p50 author→consumed lead time	Analytics lead-time widget + dashboard tile
US-301/302 delivery API	O3	# apps live · # namespaces	Saved query Tag=Delivery AND State=Done + telemetry
US-201/202 review	O4	required review bypassed = 0	Saved query RequiredReview=Yes AND PublishedWithoutReview=Yes (must return 0)
F1–F4 flow	O2	text-change deploys/week → 0	Weekly metric; CFD watched for review-column WIP

Cadence: weekly 30-min delivery review (indicators + CFD); monthly outcome review vs O1–O3 with O4 as the standing first item.

4. Value stream (one string change)

PT = hands-on; LT = elapsed incl. wait. Dominant queue: the **review-approval wait**.

Step	PT	LT
Author edits (US-101/102)	5 min	5 min
Wait: review queue (US-201)	—	~1–2 days (review-required strings only)
Reviewer approves (US-202)	5 min	5 min

Step	PT	LT
Publish (US-102)	1 min	1 min
Propagate (Redis invalidation)	~0	≤ 60s
Consumed (US-301)	~0	≤ 200ms p95

PT ≈ 11 min; LT ≈ 1–2 days, ~99% of it the review queue. Non-review strings already flow author→consumed in minutes — the queue is the whole gap between O1's promise and today.

5. Bottleneck-removal experiment

Targets the \$4 queue (review-approval wait).

- **Hypothesis:** most strings carry no legal/compliance risk, so **reviews optional-by-default** — with the `critical/legal` flag forcing required review on flagged keys — removes the review queue for the majority, without letting a required review be skipped.
- **Test:** 2 weeks, 3 pilot products, reviews optional-by-default + flag override; A/B against 3 controls on always-review. Measure author→consumed lead time (ADO widget) and the O4 guardrail query.
- **Success:** baseline p50 ≈ **1.5 days** → target ≤ **15 min** for non-review strings, ≥80% take the no-review path. Guardrail: `critical/legal` published without review = **0**; reject/hot-fix rate no worse than control. Window 2 weeks, then a roll-out decision.

Model-answer note (facilitator): O1 is user-visible (text live in minutes), not "publish endpoint shipped," and its indicator ties to the same SLOs as TCD/TMD (≤60s, ≤200ms). Every outcome carries a don't-game-it guardrail; O4 is the counter-outcome. The backlog is the same R1 slice as the rest of the folder, each story tagged to its CS. The experiment removes the actual bottleneck the value stream exposes, with a real baseline, target, guardrail, and window.